CompileX (Code Lipi)

UITS Compiler Lab project. Ekta chhoto programming language, jar function gulor nam Banglish e (Jog, Biyog, Dekhao...).

Chalanor niyom

```
g++ -std=c++17 -o compilex compilex.cpp     # compile (ekbar)
  ./compilex demo.cx                          # code chalao
  ./compilex demo.cx --tokens                 # Phase 1 er output
  ./compilex demo.cx --ast                    # Phase 2 er output (text tree)
  ./compilex functions.cx --report            # proti function er Token + Parse Tree +
  Output (report.html)
```

(Windows CMD te `./compilex` er jaygay `.\compilex` lekho.)

`--report` e `functions.cx` er proti function ekta card hoye ashe: Source, Token table, Parse Tree, Output. `functions.cx` e `# title:`, `# note:`, `# input:` comment diye card er nam, byakkha ar nao() er input dewa jay.

Nijer hate input dewar 2 ta upay

```
./compilex interactive.cx     # CompileX Calculator


  ./compilex                    # interactive mode: cx> prompt e nijer code type koro
  ./compilex interactive.cx     # program ta user ke diye input chaibe (nao diye)
```

Interactive mode er example:

```
cx> jog(2, 3)
  5
  cx> dhori marks = [70, 85, 60]
  cx> gor(marks)
  71.6667
  cx> jotokhon ... koro     # block likhle ... dekhabe, 'sesh' dile chalabe
  cx> exit
```

List e notun man jog korte: `list = list + [x]`

```
Example (interactive mode-e try koro):

  cx> dhori list = [4, 9]
  cx> dhori x = 2
  cx> list = list + [x]
  cx> list
  [4, 9, 2]
```

Shudhu C++17 compiler (g++) lage, kono library lage na. Windows e: `compilex.exe demo.cx`.

Compiler er 3 ta phase (sir ke eta bolbe)

Phase	Kaj	Code e kothay
1. Lexer	Source code ke token e bhag kore	`lexer()`
2. Parser	Token theke AST (syntax tree) banay, grammar check kore	`class Parser`
3. Interpreter	AST walk kore code chalay	`class Interpreter`

Example: `dhori a = jog(2, 3)`

- Lexer: `dhori`, `a`, `=`, `jog`, `(`, `2`, `,`, `3`, `)`
- Parser: `('let', 'a', ('call', 'jog', [2, 3]))`
- Interpreter: `jog` table e khuje 2+3 = 5, `a` te rakhe

Language er syntax

```
dhori x = 10                    # variable
  x = x + 1                       # man bodlano
  jodi x > 5 tahole ... nahole ... sesh      # condition
  jotokhon x < 20 koro ... sesh              # loop
  # eta comment
```

25 ta function (20 ta proposal er + 5 ta extra)

SL	Standard	UITS	Example	Result
1	Add	`jog`	`jog(10, 5)`	15
2	Subtract	`biyog`	`biyog(10, 5)`	5
3	Multiply	`gun`	`gun(10, 5)`	50
4	Divide	`vag`	`vag(10, 4)`	2.5
5	Modulus	`vagsesh`	`vagsesh(10, 3)`	1
6	Power	`ghat`	`ghat(2, 8)`	256
7	Average	`gor`	`gor([70, 80])`	75
8	Maximum	`boro`	`boro([3, 9, 4])`	9
9	Minimum	`choto`	`choto([3, 9, 4])`	3
10	Print	`dekhao`	`dekhao("Hi")`	Hi
11	Input	`nao`	`nao("Nam: ")`	user er input
12	Length	`lomba`	`lomba("UITS")`	4
13	Reverse	`ulta`	`ulta("UITS")`	STIU
14	Sort	`sajao`	`sajao([3, 1, 2])`	[1, 2, 3]
15	Search	`khojo`	`khojo([5, 7], 7)`	1 (position, na pele -1)
16	Count	`gono`	`gono([1, 2, 1], 1)`	2
17	Compare	`melao`	`melao(5, 9)`	-1 (choto), 0 (soman), 1 (boro)
18	Swap	`bodol`	`bodol(x, y)`	x, y er man adol-bodol
19	Delete	`mucha`	`mucha(list, 60)`	list theke 60 muche jay
20	Clear	`saf`	`saf(list)`	list khali hoye jay
21	Sum	`mot`	`mot([10, 20, 30])`	60
22	Square	`borgo`	`borgo(5)`	25
23	Square root	`mul`	`mul(16)`	4
24	Round	`gol`	`gol(3.6)`	4
25	Absolute	`porom`	`porom(-7)`	7

Notun function jog korar upay

`Interpreter::call()` er vitore ekta `if` block add korlei hoy:

```
if (name == "borgo") { need(1); return mkNum(num(0) * num(0)); }   // borgo(5) = 25
```

Error hole line number soho message dekhay, jemon: `Error (line 3): Shunno diye vag kora jay na`